```
#task1
def convert_units(conversion_type, value, direction):
    """
    Converts between different measurement units.

    Parameters:
    conversion_type (str): Type of conversion ('length', 'weight', 'volume')
    value (float): Numeric value to convert
    direction (int): Conversion direction option

    Returns:
    float: Converted value
    """

    if conversion_type == "length":
        if direction == 1:  # meters to feet
            return value * 3.28084
        elif direction == 2:  # feet to meters
            return value / 3.28084

    elif conversion_type == "weight":
        if direction == 1:  # kg to pounds
            return value * 2.20462
        elif direction == 2:  # pounds to kg
            return value / 2.20462

    elif conversion_type == "volume":
        if direction == 1:  # liters to gallons
            return value * 0.264172
        elif direction == 2:  # gallons to liters
            return value / 0.264172

    else:
        raise ValueError("Unsupported conversion type")


try:
    print("Choose conversion type: length / weight / volume")
    ctype = input("Enter type: ").lower()

    print("1. First unit to second unit")
    print("2. Second unit to first unit")
    direction = int(input("Enter choice (1 or 2): "))

    value = float(input("Enter value: "))

    result = convert_units(ctype, value, direction)

    print("Converted value:", result)

except ValueError:
    print("Invalid input. Please enter numeric values.")
```

```
Choose conversion type: length / weight / volume
Enter type: length
1. First unit to second unit
2. Second unit to first unit
Enter choice (1 or 2): 1
Enter value: 7
Converted value: 22.96588
```

```
#task2
def find_sum(numbers):
    """
    Returns the sum of numbers in a list.
    """
    return sum(numbers)


def find_average(numbers):
    """
    Returns the average of numbers in a list.
    """
    return sum(numbers) / len(numbers)
```

```
def find_max(numbers):
    """
    Returns the maximum number in the list.
    """
    return max(numbers)


def find_min(numbers):
    """
    Returns the minimum number in the list.
    """
    return min(numbers)


try:
    nums = input("Enter numbers separated by spaces: ")
    numbers = [float(x) for x in nums.split()]

    if len(numbers) == 0:
        raise ValueError("List is empty")

    print("Choose operation: sum / average / max / min")
    op = input("Enter operation: ").lower()

    if op == "sum":
        print(find_sum(numbers))
    elif op == "average":
        print(find_average(numbers))
    elif op == "max":
        print(find_max(numbers))
    elif op == "min":
        print(find_min(numbers))
    else:
        print("Invalid operation")

except ValueError:
    print("Invalid input. Enter numeric values.")
```

```
Enter numbers separated by spaces: 3
Choose operation: sum / average / max / min
Enter operation: sum
3.0
```

```
#List manuplation
def extract_every_other(lst):
    return lst[::2]


def get_sublist(lst, start, end):
    return lst[start:end+1]


def reverse_list(lst):
    return lst[::-1]


def remove_first_last(lst):
    return lst[1:-1]


def get_first_n(lst, n):
    return lst[:n]


def get_last_n(lst, n):
    return lst[-n:]


def reverse_skip(lst):
    return lst[-2::-2]
```

```
#Nested list
#1
def flatten(lst):
    result = []
    for i in lst:
```

```
        if isinstance(i, list):
            result.extend(flatten(i))
        else:
            result.append(i)
    return result
```

```
#2
def access_nested_element(lst, indices):
    element = lst
    for i in indices:
        element = element[i]
    return element
```

```
#3
def sum_nested(lst):
    total = 0
    for i in lst:
        if isinstance(i, list):
            total += sum_nested(i)
        else:
            total += i
    return total
```

```
#4
def remove_element(lst, elem):
    new_list = []
    for i in lst:
        if isinstance(i, list):
            new_list.append(remove_element(i, elem))
        elif i != elem:
            new_list.append(i)
    return new_list
```

```
#5
def find_max(lst):
    maximum = float("-inf")

    for i in lst:
        if isinstance(i, list):
            maximum = max(maximum, find_max(i))
        else:
            maximum = max(maximum, i)

    return maximum
```

```
#6
def count_occurrences(lst, elem):
    count = 0

    for i in lst:
        if isinstance(i, list):
            count += count_occurrences(i, elem)
        elif i == elem:
            count += 1

    return count
```

```
#7
def deep_flatten(lst):
    result = []

    for i in lst:
        if isinstance(i, list):
            result.extend(deep_flatten(i))
        else:
            result.append(i)

    return result
```

```
#8
def average_nested(lst):
```

```
    flat = deep_flatten(lst)

    return sum(flat) / len(flat)
```

```
#array creation
#1
import numpy as np
arr1 = np.empty((2,2))
print(arr1)
```

```
[[4.17560493e-315 0.00000000e+000]
 [4.94065646e-324             nan]]
```

```
#2
arr2 = np.ones((4,2))
print(arr2)
```

```
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]
```

```
#3
arr3 = np.full((3,3), 7)
print(arr3)
```

```
[[7 7 7]
 [7 7 7]
 [7 7 7]]
```

```
#4
arr4 = np.zeros_like(arr2)
print(arr4)
```

```
[[0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]]
```

```
#5
arr5 = np.ones_like(arr2)
print(arr5)
```

```
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]
```

```
#6
new_list = [1,2,3,4]
arr6 = np.array(new_list)
print(arr6)
```

```
[1 2 3 4]
```

```
#Problem2
#1
arr = np.arange(10,50)
print(arr)
```

```
[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49]
```

```
#2
matrix = np.arange(9).reshape(3,3)
print(matrix)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

```
#3
identity = np.eye(3)
```

```
print(identity)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
#4
rand_arr = np.random.random(30)
print(rand_arr)

mean_val = rand_arr.mean()
print("Mean:", mean_val)
```

```
[0.36887585 0.28463599 0.07977279 0.72850284 0.6029291  0.68346119
 0.77443018 0.51787181 0.66731647 0.94150552 0.54420976 0.0398217
 0.27549751 0.00966568 0.51967312 0.84037616 0.3887755  0.23186407
 0.66394655 0.89667641 0.6960311  0.84372571 0.09660861 0.1993999
 0.90045886 0.00444368 0.43041713 0.4539674  0.5204921  0.79833193]
Mean: 0.5001228205081143
```

```
#5
arr = np.random.random((10,10))

print("Minimum:", arr.min())
print("Maximum:", arr.max())
```

```
Minimum: 0.0005850518284027384
Maximum: 0.973741890480183
```

```
#6
arr = np.zeros(10)
arr[4] = 1
print(arr)
```

```
[0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
```

```
#7
arr = np.array([1,2,0,0,4,0])
reversed_arr = arr[::-1]
print(reversed_arr)
```

```
[0 4 0 0 2 1]
```

```
#8
arr = np.ones((5,5))
arr[1:-1,1:-1] = 0
print(arr)
```

```
[[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]
```

```
#9
checker = np.zeros((8,8), dtype=int)

checker[1::2, ::2] = 1
checker[::2, 1::2] = 1

print(checker)
```

```
[[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]]
```

```
#problem 3:
x = np.array([[1,2],[3,5]])
y = np.array([[5,6],[7,8]])
```

```
v = np.array([9,10])
w = np.array([11,12])
```

```
#1
print(x + y)
```

```
[[ 6  8]
 [10 13]]
```

```
#2
print(x - y)
```

```
[[-4 -4]
 [-4 -3]]
```

```
#3
print(x * 3)
```

```
[[ 3  6]
 [ 9 15]]
```

```
#4
print(np.square(x))
```

```
[[ 1  4]
 [ 9 25]]
```

```
#5
print("v dot w =", np.dot(v,w))

print("x dot v =", np.dot(x,v))

print("x dot y =\n", np.dot(x,y))
```

```
v dot w = 219
x dot v = [29 77]
x dot y =
 [[19 22]
 [50 58]]
```

```
#6
print(np.concatenate((x,y), axis=0))
```

```
[[1 2]
 [3 5]
 [5 6]
 [7 8]]
```

```
#7
np.concatenate((x,v))
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_285/3136158255.py in <cell line: 0>()
      1 #7
----> 2 np.concatenate((x,v))

ValueError: all the input arrays must have same number of dimensions, but the array at index 0 has 2 dimension(s) and the array
at index 1 has 1 dimension(s)
```

```
#matrix algebra
#1
A = np.array([[1,2],[3,4]])

A_inv = np.linalg.inv(A)

result = np.dot(A, A_inv)

print(result)
```

```
[[1.0000000e+00 0.0000000e+00]
 [8.8817842e-16 1.0000000e+00]]
```

```
#2
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

AB = np.dot(A,B)
BA = np.dot(B,A)

print("AB =\n",AB)
print("BA =\n",BA)
```

```
AB =
 [[19 22]
 [43 50]]
BA =
 [[23 34]
 [31 46]]
```

```
#3

AB = np.dot(A,B)

left = AB.T
right = np.dot(B.T, A.T)

print("Left side:\n", left)
print("Right side:\n", right)
```

```
Left side:
 [[19 43]
 [22 50]]
Right side:
 [[19 43]
 [22 50]]
```

```
#10.2
import numpy as np
import time
size = 1000000

a = list(range(size))
b = list(range(size))

start = time.time()

c = [a[i] + b[i] for i in range(size)]

end = time.time()

print("Python list addition time:", end-start)
```

```
Python list addition time: 0.11594367027282715
```

```
a = np.arange(size)
b = np.arange(size)

start = time.time()

c = a + b

end = time.time()

print("NumPy addition time:", end-start)
```

```
NumPy addition time: 0.035341739654541016
```

```
#2
start = time.time()

c = [a[i] * b[i] for i in range(size)]

end = time.time()

print("Python multiplication time:", end-start)
```

```
Python multiplication time: 0.6516289710998535
```

```
start = time.time()

c = a * b

end = time.time()

print("NumPy multiplication time:", end-start)
```

```
NumPy multiplication time: 0.02041482925415039
```

```
#3
start = time.time()

dot = sum(a[i]*b[i] for i in range(size))

end = time.time()

print("Python dot product time:", end-start)
```

```
Python dot product time: 0.3603966236114502
```

```
start = time.time()

dot = np.dot(a,b)

end = time.time()

print("NumPy dot product time:", end-start)
```

```
NumPy dot product time: 0.002991199493408203
```

```
#4
import random

size = 100

A = [[random.random() for _ in range(size)] for _ in range(size)]
B = [[random.random() for _ in range(size)] for _ in range(size)]
```

```
A = np.random.rand(1000,1000)
B = np.random.rand(1000,1000)

start = time.time()

C = np.dot(A,B)

end = time.time()

print("NumPy matrix multiplication time:", end-start)
```

```
NumPy matrix multiplication time: 0.07505345344543457
```